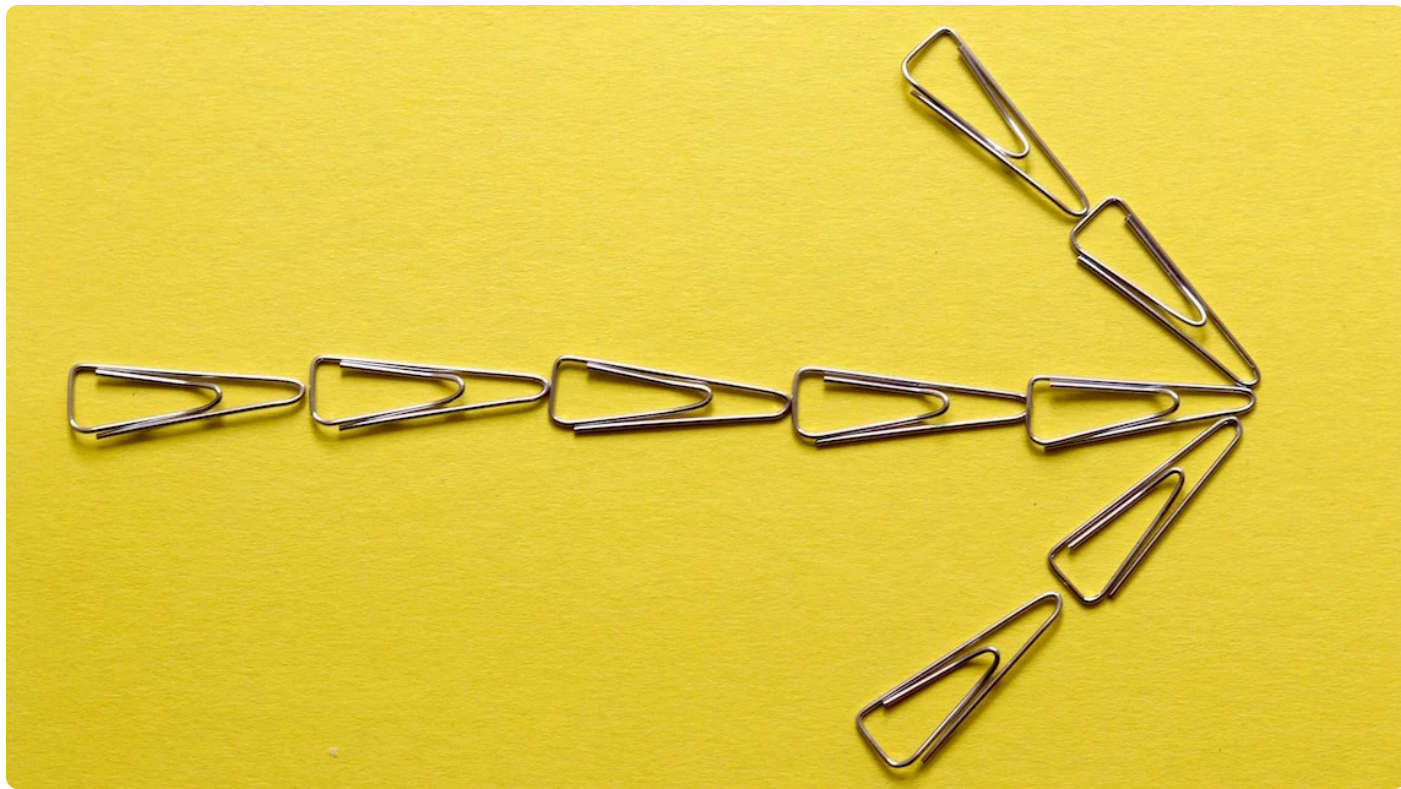


15 | 精细化成本管理：两大工具助力降本增效

潘野 · 云原生基础架构实战课



你好，我是潘野。

之前的课程里，我们学习了扩缩容的相关技术。不过即便采用了这样的技术，可能基础架构部门仍然要为公有云逐月增加的费用头疼。

还记得我们 [第 12 讲](#) 的例子么？用户为 Redis 配置了超大内存的机器，但实际利用率仅为 20%。这种配置不匹配的情况，触发了 Cluster Autoscaler 的降配操作，最终导致 Redis 集群崩溃。这个案例表明，尽管自动扩缩容技术可以帮助管理资源，但如果没有恰当的配置和监控，同样可能浪费资源，影响业务稳定性。

为了解决这样的问题，我们就需要制定更精细地成本管理策略。今天这一讲，我们就来聊聊成本管理这个话题，帮助你的团队降本增效，把钱花在刀刃上。

公有云成本优化策略

我们先从一个 EC2 实例管理的真实案例出发，了解一下资源精细化管理可以从哪些角度入手。

我经历过这样一个情况：某个月公有云的费用增长了 30%，其中 EC2 实例费用占比最高，达到 70%。持续增高的公有云成本让我们非常头疼。经过细致的盘点和分析，我们识别出 EC2 实例花费高昂费用，背后可能是这些原因。

1. 未充分利用弹性伸缩功能。
2. 闲置实例未及时回收。
3. 实例规格过高。
4. 没有选择适合的实例类型

我相信这类问题很多公司都会遇到。为了解决这些问题，我们主要有这几种应对思路。

1. 精确资源评估：在应用上线前进行详细的资源需求评估，以减少上线后因频繁扩缩容引起的应用不稳定，从而间接节约成本。
2. 分析云成本构成：我们可以分析 EC2 实例的费用构成，并识别出闲置和利用率低的实例。
3. 资源申请审计：在申请资源时进行成本预估，如果预见到较高的花费，应进一步分析以避免资源过度申请。
4. 运行后的资源评估：应用上线后，定期评估资源使用率，并根据实际使用情况调整资源配置。

优化资源配置

刚刚我们明确了解决思路，但你应该很快就会想到，想要做成本分析和预测，离不开相关工具的帮助。

市面上有多种成本优化工具，可供我们选择。这些工具提供成本分析、预算管理、资源优化等功能，帮助我们有效地降低云成本。接下来，我们重点学习 Infracost 和 OpenCost 这两个工具，它们能为云资源管理提供实时的成本分析和预测。

Infracost

Infracost 是一个开源的用于估算和监控云计算资源成本的工具。

在技术团队把开发好的应用部署到 AWS、GCP 等云平台之前，Infracost 可以帮助我们理解和控制基础设施的成本。

而在业务上线之后，Infracost 能够监控云计算资源成本，并提供详细的成本报告，给出成本优化建议。我们可以使用这些报告来跟踪成本趋势，并识别潜在的成本节省机会。

那么 Infracost 是如何估算成本的呢？主要是通过四大步骤来实现的。

第一步，**解析基础设施代码**。Infracost 支持多种 IAC 工具，比如我们前面讲过的 Terraform、Pulumi 等。它首先会解析这些工具的配置文件，识别将要创建、修改或删除的资源。

第二步，**获取定价信息**。Infracost 从云服务提供商的定价 API 获取资源的价格信息。其中包括各种服务的使用费用，例如计算实例、存储、网络等成本。

第三步，**计算成本**。Infracost 会基于解析出的资源配置和获取的定价信息，计算出每个资源的预估成本。它既可以提供单个资源的成本，也可以计算整个项目的总成本。

最后一步是**生成报告**。Infracost 能够生成详细的成本报告，显示每种资源的成本以及总成本。这些报告可以是命令行输出，也可以是更丰富的格式（如 HTML 或 JSON）。

接下来，为了帮助你加深理解。我们用课程中启动 EKS 集群的 Terraform 代码作为样例，来看看怎么使用 Infracost。

首先，我们需要在本机上安装 infracost，安装方式支持 Mac、Linux、Windows 还有 Docker。这里我选择安装在 Mac 上，你可以按照 [🔗 官方文档](#) 自行选择安装方式。

Mac 上使用 brew 就可以安装 Infracost。

 复制代码

```
1 >>> brew install infracost
```

接下来，我们通过命令行 `infracost auth login` 来登陆 Infracost 的网站。Infracost 会给你下发一个 API Key，使用这个 API 密钥，Infracost 命令行就可以访问公共云价格 API，将 Terraform 里的资源的价格拉取回来进行计算。

从下面的命令输出我们可以看出，成功登陆之后。我们在 Terraform 目录里跑一下 `infracost breakdown --path=.` 这个命令，就可以获得预估的费用了。

 复制代码

```
1 >>> infracost auth login
2 We're redirecting you to our log in page, please complete that,
3 and return here to continue using Infracost.
4
5 If the redirect doesn't work, either:
6 - Use this URL:
7   https://dashboard.infracost.io/login?xxxxx_version=v0.10.33&os=darwin&utm_sou
8
9 - Or log in/sign up at https://dashboard.infracost.io, copy your API key
10   from Org Settings and run `infracost configure set api_key MY_KEY`
11
12 Waiting...
13
14 The API key was saved to /Users/pye/.config/infracost/credentials.yml
15
16 Your account has been authenticated. Run Infracost on your Terraform project by r
17
18   infracost breakdown --path=.
```

接下来，我们将代码 <https://github.com/cloudnative-automation/eks-cluster.git> 下载到本地之后，运行 `infracost breakdown --path=.` 这个命令。

运行之后，Infracost 会先去初始化，下载 terraform 代码需要的模块，检索云资源的价格，然后输出 730 小时预估的费用。从后面这张图就可以看出各个资源的预估费用的详情，我们的 EKS 集群一个月大约需要花费 167 美元。

```
Evaluating Terraform directory at .
✓ Downloading Terraform modules
✓ Evaluating Terraform directory
✓ Retrieving cloud prices to calculate costs

Project: cloudnative-automation/eks-cluster

Name                                                                 Monthly Qty  Unit              Monthly Cost
-----
module.eks.aws_cloudwatch_log_group.this[0]
├─ Data ingested                                                    Monthly cost depends on usage: $0.50 per GB
├─ Archival Storage                                                Monthly cost depends on usage: $0.03 per GB
└─ Insights queries data scanned                                   Monthly cost depends on usage: $0.005 per GB

module.eks.aws_eks_cluster.this[0]
├─ EKS cluster                                                    730  hours              $73.00

module.eks.module.eks_managed_node_group["one"].aws_eks_node_group.this[0]
├─ Instance usage (Linux/UNIX, on-demand, t3.small)              1,460  hours              $30.37

module.eks.module.eks_managed_node_group["two"].aws_eks_node_group.this[0]
├─ Instance usage (Linux/UNIX, on-demand, t3.medium)              730  hours              $30.37

module.vpc.aws_nat_gateway.this[0]
├─ NAT gateway                                                    730  hours              $32.85
└─ Data processed                                                Monthly cost depends on usage: $0.045 per GB

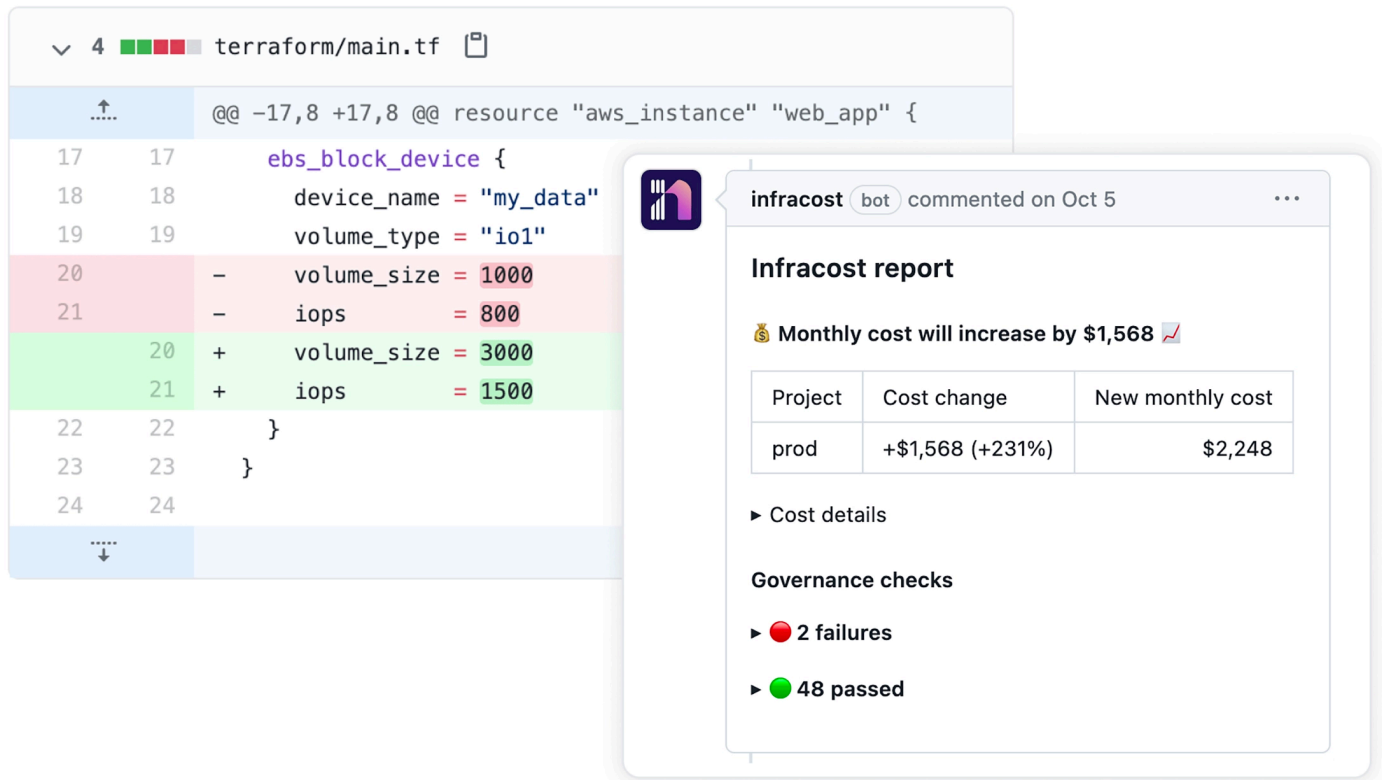
OVERALL TOTAL                                                                 $166.59

55 cloud resources were detected:
• 5 were estimated, 4 of which include usage-based costs, see https://infracost.io/usage-file
• 50 were free, rerun with --show-skipped to see details
```

Project	Monthly cost
cloudnative-automation/eks-cluster	\$167

另外，Infracost 还可以集成到持续集成 / 持续部署（CI/CD）流程中，自动执行成本估算。这样，我们就可以在代码更改时自动获得成本反馈，从而做出成本更低、效益更高的决策。

我再举个例子，当我们提交了一个 Terraform 的代码改动，比如改动了一个资源类型，将磁盘的 size 从 1000G 改成了 3000G。这时 Infracost 也可以帮我们预估一个月可能要多花费多少钱，然后在 CI 中展示出来。你可以参考后面的图片看一下。



图片源自官网 <https://github.com/infracost/actions/blob/master/.github/assets/pull-request.png>

Opencost

Infracost 的主要亮点是云上资源的成本预估，那么我们想知道一个应用浪费了多少的资源应该怎么办呢？这时候我们就需要用到 OpenCost 这个工具。

OpenCost 的前身是 Kubecost，现在是 CNCF 中的一个沙盒项目，主要是用于监控 Kubernetes 相关的云成本，可以根据命名空间、应用标签和应用部署这些不同维度来分析 Kubernetes 集群成本。

它的工作原理和 Infracost 的类似。

首先，Opencost 通过节点上的代理收集有关节点和 Pod 资源使用情况的数据，比如 CPU、内存和持久性存储。然后连接到云提供商的账单 API，以获取与集群相关的实际成本和定价数据。OpenCost 会结合资源使用数据与定价信息，计算出每个工作负载、命名空间、部署等方面的成本。最终，通过一个 UI 仪表板，直观展示出 Kubernetes 中资源实体的成本如何随着时间变化。

另外，它还支持生成详细的成本报告，并且可以配置成本自动预警功能，即当成本超过预定义阈值时发送警报。

了解原理之后，我们来看看怎么使用 OpenCost。Opencost 依赖 Prometheus 来收集数据信息，所以我们需要在集群里先安装 Prometheus。

官网提供了安装 Prometheus 的方式，我贴在下面，你根据命令安装即可。

 复制代码

```
1 helm install prometheus --repo https://prometheus-community.github.io/helm-charts
2   --namespace prometheus-system --create-namespace \
3   --set prometheus-pushgateway.enabled=false \
4   --set alertmanager.enabled=false \
5   -f https://raw.githubusercontent.com/opencost/opencost/develop/kubernetes/prometheus
```

如果你的集群里已经安装好了 Prometheus，可以直接进入下一步，运行下面这个命令安装 OpenCost。

 复制代码

```
1 kubectl apply --namespace opencost -f https://raw.githubusercontent.com/opencost/opencost/
```

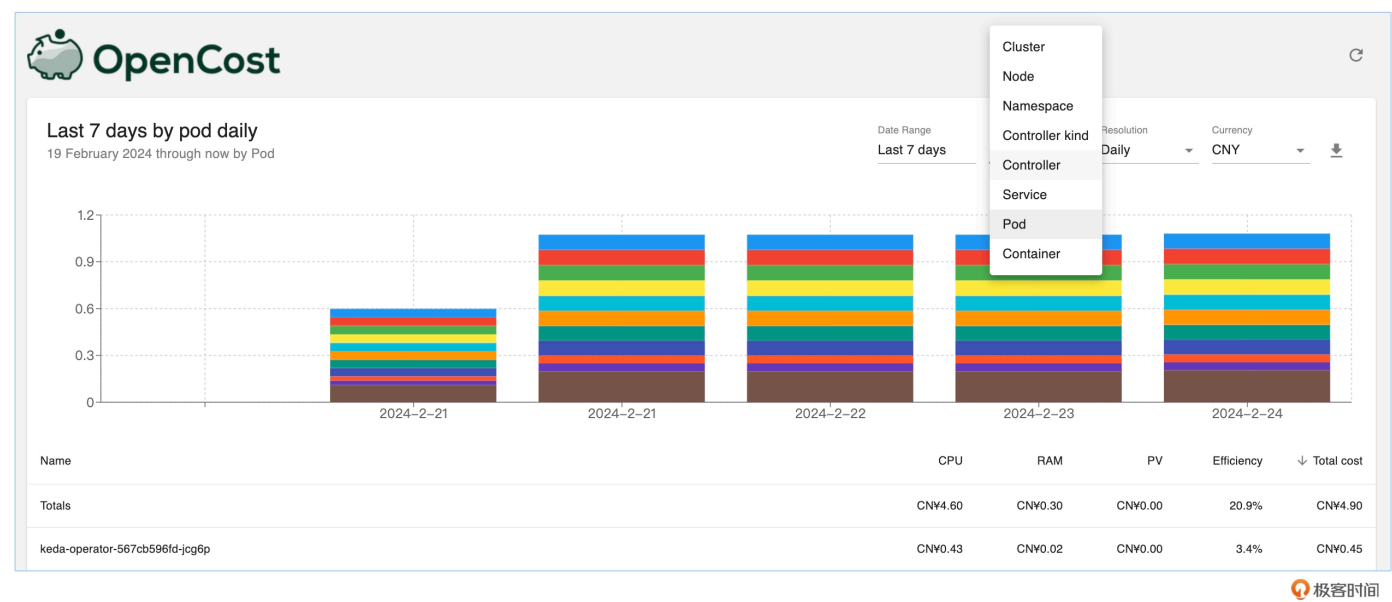
这个配置文件中读取的 Prometheus 地址是 `prometheus-server.prometheus-system.svc`，你需要按照自己的情况，将 Deployment 中 Prometheus 的地址改成你自己的 Prometheus 服务器地址。

安装完成之后，我们可以用 port-forward 或者 ingress 的方式来访问 OpenCost 的 UI，这里我使用的是 port-forward 的方式。

 复制代码

```
1 kubectl port-forward --namespace opencost service/opencost 9003 9090
```


我们在浏览器中打开 <http://localhost:9090>，就可以看到 Opencost 的 UI 了。后面这张图就是界面的样子，我们可以基于 Node、Namespace、Pod 和 Container 等不同的维度来分析资源利用率，同时也能计算出当前的资源花费了多少钱。



选择合适的资源类型

刚才，我们重点介绍了 Infracost 和 OpenCost 这两个成本分析和预测工具。接下来，我再给你分享一些帮助我们节约云成本的实用技巧。

选择合适的实例类型

公有云厂商虽然投资了大量的 IT 基础设施。不过，这些基础设施不可能一直处于满负荷状态，必然会存在闲置资源。这不仅会造成浪费，还会产生一定的维护成本。

因此，厂商也要提升资源的利用率。公有云厂商为了将闲置资源利用起来，推出了一种可抢占式实例，这种实例的价格比按需实例低得多，通常可享受 50% 以上的折扣，甚至高达 90%。

需要注意的是，这种可抢占式的实例主打的是便宜，但是不保证可用性。如果此时公有云内部的资源不够用了，公有云会主动回收掉实例，将资源分配给出价高的租户。

使用可抢占式实例的场景主要在以下几个方面。

容器编排：例如 Kubernetes 集群；

批处理作业：例如数据分析、机器学习训练；

测试和开发：例如代码编译、单元测试、性能测试等情况，其共性是**应用具备容错机制，能够应对实例被回收的情况。**

各个厂商都有这种可抢占式实例。AWS 上叫 Spot instances，Azure 上叫低成本虚拟机，GCP 上叫预留实例，阿里云上叫抢占实例。

所以在云原生场景中，我们可以为 Kubernetes 集群配置可抢占实例，利用 Kubernetes 良好的容器管理功能，在实现高可用的同时，大幅度降低成本。

便宜的实例类型一定省钱么

下面我们讨论一下，实例类型的选择问题。我想和你分享一个我在 AWS 中遇到的真实案例。

在 AWS 的 EKS 中，我们最初在 EC2 实例上选择使用 Intel CPU。在某一次业务迁移中，我们将 Intel CPU 的 EC2 实例换成了 AMD CPU，CPU 的核心数目保持不变，因为同样配置的情况下，AMD CPU 的实例比 Intel CPU 要稍稍便宜一些。

在业务高峰期，集群流量入口 nginx-ingress 所在的 EC2 实例，CPU 利用率达到 100%，无法响应新请求。尽管 Cluster Autoscaler 触发了水平扩容，将 nginx-ingress 的 EC2 实例数量扩大了三倍，但没有解决问题，nginx-ingress EC2 实例的 CPU 仍然保持在 100%。**也就是用了更多的机器，花了更多的钱，但性能反而下降了。**

后来，我们经过仔细对比分析，发现问题出在 AMD CPU 上。我们从 [Intel 的官方网站](#)上得到这样的一个信息，在 AWS 上使用 Intel CPU 的 C6i 实例，在处理加解密场景中，具有加密加速功能的 Intel 快速辅助技术，可以大大加速 Nginx 中 TLS 的加解密性能，其性能大约是使用 AMD CPU 的 C6a 的三倍。

所以这种场景下，尽管看起来使用 Intel CPU 的机型比 AMD CPU 机型要略微贵一点点，但是性能却有三倍的差距。那么在这种情况下，从性能和成本效益角度来看，选择性能更强的 Intel CPU 反而是更经济的选择。

总结

今天，我们重点学习了两个成本优化工具，分别是 Infracost 和 Opencost。使用这些工具，可以帮我们预估资源申请的费用，在业务运营中更科学地计算成本，找到更多的降本增效机会。

除了这两个工具之外，我们还学习了云上一种收费低廉的资源——可抢占式实例。这是一种显著降低公有云成本的方式。不过使用之前，需要了解它的适用场景（容器编排、批处理作业、测试和开发），再决定是否可以引入。

最后，我为你分享了一个真实的案例。这个例子主要是提醒你不要被表象蒙蔽，一定要结合具体业务场景和实际情况，才有可能制定出更合理的成本控制方案。

思考题

回顾一下这个模块的内容，应用层面的水平扩缩容 HPA，垂直扩缩容 VPA，集群层面的扩缩容 Karpenter 与 Cluster Autoscaler，请你总结一下它们适用的场景与缺陷分别是什么。

欢迎在留言区和我交流互动，如果这节课对你有启发，别忘了分享给身边更多朋友。

AI智能总结

1. 精细化成本管理对于公有云资源的优化至关重要，需要从未充分利用弹性伸缩功能、闲置实例未及时回收、实例规格过高、没有选择适合的实例类型等方面入手。
2. 成本管理工具Infracost和OpenCost能够帮助理解和控制基础设施的成本，通过解析基础设施代码、获取定价信息、计算成本和生成报告等步骤来实现成本估算和监控。
3. 选择合适的实例类型和资源类型是关键，可抢占式实例是一种降低成本的方式，但需注意其适用场景和可用性。
4. 在选择实例类型时，需要综合考虑性能和成本效益，避免仅以价格为标准而忽视性能差异。
5. Infracost和OpenCost工具能够为云资源管理提供实时的成本分析和预测，帮助有效地降低云成本。
6. 应用层面的水平扩缩容HPA、垂直扩缩容VPA，集群层面的扩缩容Karpenter与Cluster Autoscaler，适用的场景与缺陷需要结合具体情况进行综合考量。

全部留言 (1)

最新 精选



Starduster

2024-07-20 来自日本

老师你好，我有一个疑问，Intel QAT技术是需要自行更换SSL engine或是使用Intel SDK才能生效，nginx无法out of box使用，文中提到的性能问题应该不是QAT导致的吧？

